# LAPORAN PRAKTIKUM PEKAN 3

# STRUKTUR DATA


**Disusun Oleh:**

Syasya Halwa Gazwani

(2511531018)

**Dosen Pengampu:**

Dr. Wahyudi, S.T, M.T

**Asisten Praktikum:**

Muhammad Galid Alvero

**DEPARTEMEN INFORMATIKA**

**FAKULTAS TEKNOLOGI INFORMASI**

**UNIVERSITAS ANDALAS**

**PADANG**


**2026**

# KATA PENGANTAR

Assalamu'alaikum warahmatullahi wabarakatuh,

Segala puji bagi Allah SWT yang telah memberikan kesempatan serta kemudahan kepada penulis untuk dapat menyelesaikan Laporan Praktikum pada mata kuliah Struktur Data, sehingga Laporan Praktikum ini dapat dikumpulkan dengan tepat waktu. Atas rahmat dan karunianya Laporan Praktikum dapat terselesaikan dengan baik. Sholawat serta salam juga penulis sampaikan kepada baginda tercinta yaitu Nabi Muhammad SAW. yang kita nantikan syafa'atnya di akhirat nanti. Laporan Praktikum ini bertujuan untuk menambah wawasan para pembaca untuk lebih memperdalam ilmu yang ada pada makalah ini.

Dalam penyusunan Laporan Praktikum ini, penulis mengalami banyak kesulitan dan penulis menyadari bahwa Laporan Praktikum ini masih jauh dari kesempurnaan. Untuk itu, penulis sangat mengharapkan kritik dan saran yang membangun demi kesempurnaan pada Laporan Praktikum ini. Penulis mengucapkan terima kasih kepada bapak Dr. Wahyudi, S.T, M.T. selaku dosen mata kuliah Struktur Data yang telah memberikan tugas ini sehingga dapat menambah pengetahuan dan wawasan sesuai dengan mata kuliah yang penulis tekuni. Penulis juga mengucapkan terima kasih kepada semua pihak yang telah membagi sebagian pengetahuannya sehingga penulis dapat menyelesaikan Laporan Praktikum ini.

Padang, April 2026

**Penulis**

# DAFTAR ISI

**KATA PENGANTAR** ..... i

**DAFTAR ISI** ..... ii

**BAB I PENDAHULUAN** ..... 1

**1.1 Latar Belakang** ..... 1

**1.2 Tujuan** ..... 2

**1.3 Manfaat Praktikum** ..... 2

**BAB II PEMBAHASAN** ..... 3

**2.1 stackArray_2511531018** ..... 3

**2.1.1 Kode Program** ..... 3

**2.1.2 Langkah Kerja** ..... 4

**2.2 stackArrayDriver_2511531018** ..... 5

**2.2.1. Kode Program** ..... 5

**2.2.2. Langkah Kerja** ..... 6

**2.2.3. Output** ..... 7

**2.3 StackPostFix_2511531018** ..... 7

**2.3.1 Kode Program** ..... 7

**2.3.2 Langkah Kerja** ..... 8

**2.3.3. Output** ..... 9

**2.4 NilaiMaksimum_2511531018** ..... 9

**2.4.1 Kode Program** ..... 9

**2.4.2 Langkah Kerja** ..... 10

**2.4.3 Ouput** ..... 11

**2.5 SiswaStack_2511531018** ..... 12

**2.5.1 Kode Program** ..... 12

**2.5.2 Langkah Kerja** ..... 13

**2.5.3 Output** ..... 15

**2.6 Tugas** ..... 15

**2.6.1 Kode Program Website_2511531018** ..... 15

**2.6.2 Kode Program Browser_2511531018** ..... 16

**2.6.3 Output** ..... 18

**BAB III PENUTUP** ..... 20

**KESIMPULAN** ..... 20

**DAFTAR PUSTAKA** ..... 21

# BAB I PENDAHULUAN

## 1.1 Latar Belakang

Dalam dunia pemrograman, pengolahan data yang berkaitan dengan waktu merupakan salah satu kebutuhan yang cukup penting, terutama pada aplikasi seperti sistem penjadwalan, absensi, pengingat, hingga perhitungan durasi suatu kegiatan. Waktu umumnya direpresentasikan dalam bentuk jam, menit, dan detik yang memiliki batasan nilai tertentu, yaitu jam antara 0 hingga 23, serta menit dan detik antara 0 hingga 59. Jika pengolahan data waktu dilakukan secara manual menggunakan variabel biasa tanpa adanya struktur yang jelas, maka hal tersebut dapat menimbulkan berbagai permasalahan, seperti kesalahan dalam validasi data, kesulitan dalam melakukan operasi perhitungan waktu, serta kurang efisien dalam pengelolaan kode program.

Seiring dengan berkembangnya kebutuhan akan pengolahan data yang lebih kompleks dan terstruktur, diperlukan suatu pendekatan yang mampu mengorganisasi data dan fungsi secara lebih baik, yaitu dengan menggunakan konsep pemrograman berorientasi objek (Object-Oriented Programming/OOP). Dengan pendekatan ini, data dan operasi yang berkaitan dapat dikemas dalam satu kesatuan yang disebut class. Melalui class, pengolahan waktu dapat dilakukan dengan lebih sistematis, karena setiap atribut dan method yang berkaitan dengan waktu berada dalam satu struktur yang terorganisir.

Berdasarkan hal tersebut, dibuatlah sebuah class yang merepresentasikan waktu dalam bentuk jam, menit, dan detik. Class ini tidak hanya berfungsi untuk menyimpan data waktu, tetapi juga menyediakan berbagai method untuk melakukan validasi, konversi waktu ke dalam satuan detik, serta operasi aritmatika seperti penjumlahan dan pengurangan waktu. Selain itu, class ini juga dilengkapi dengan fungsi untuk membandingkan dua waktu dan menampilkan waktu dalam format yang mudah dibaca. Dengan adanya class ini, diharapkan proses pengolahan data waktu menjadi lebih mudah, akurat, dan

efisien, serta dapat digunakan kembali dalam berbagai kebutuhan pemrograman lainnya.

## 1.2 Tujuan

1.2.1 Merepresentasikan waktu (jam, menit, detik) dalam bentuk class yang terstruktur.

1.2.2 Memastikan nilai waktu valid sesuai batas (jam 0–23, menit dan detik 0–59).

1.2.3 Menyediakan fungsi untuk konversi waktu ke detik dan sebaliknya.

1.2.4 Memudahkan operasi aritmatika waktu seperti penjumlahan dan pengurangan waktu.

1.2.5 Menyediakan fungsi untuk membandingkan dua waktu.

1.2.6 Menampilkan waktu dalam format yang rapi dan mudah dibaca.

## 1.3 Manfaat Praktikum

1.3.1 Mempermudah pengolahan data waktu dalam berbagai aplikasi.

1.3.2 Mengurangi kesalahan input dengan adanya validasi waktu.

1.3.3 Mempercepat proses perhitungan waktu seperti durasi atau selisih waktu.

1.3.4 Dapat digunakan kembali (reusable) dalam program lain tanpa membuat ulang kode.

1.3.5 Membantu memahami konsep pemrograman berorientasi objek (OOP).

1.3.6 Membuat kode program lebih rapi, terstruktur, dan mudah dipelihara.

# BAB II PEMBAHASAN

## 2.1 stackArray_2511531018

### 2.1.1 Kode Program

```
package pekan3_2511531018;

public class stackArray_2511531018 {
   static final int MAX_1018 = 1000;
   int top_1018;
   int a_1018[] = new int[MAX_1018];
   boolean isEmpty_1018()
   {
         return (top_1018 < 0);

   }
   stackArray_2511531018()
   {
         top_1018 = -1;
   }
   boolean push_1018(int x)
   {
         if (top_1018 >= (MAX_1018 - 1)) {
               System.out.println("Stack overflow");
               return false;
         }
         else {
               a_1018[++top_1018] = x;
               System.out.println(x + " dimasukkan dalam
stack");
               return true;
         }
   }
   int pop_1018()
   {
         if (top_1018 < 0) {
               System.out.println("Stack Underflow");
               return 0;
         }
         else {
               int x = a_1018[top_1018--];
               return x;
         }
   }
   int peek_1018()
   {
         if (top_1018 < 0) {
               System.out.println("Stack Underflow");
               return 0;
         }
         else {
               int x = a_1018[top_1018];
               return x;
         }
   }
```

```
    void print() {
        for(int i = top_1018; i>-1; i--) {
            System.out.print(" "+ a_1018[i]);
        }

    }
}
```

Gambar 2.1.1 Kode program stackArray_2511531018.

### 2.1.2 Langkah Kerja

2.1.2.1 Max_1018 = 1000 => kapasitas maksimal stack (isi maksimal 1000 data), top_1018 => petunjuk posisi elemen paling atas di stack, dan a_1018[] => array untuk menyimpan data stack.

```
public class stackArray_2511531018 {
  static final int MAX_1018 = 1000;
  int top_1018;
  int a_1018[] = new int[MAX_1018];
```

Gambar 2.1.2.1 Deklarasi *class* dan variabel.

2.1.2.2 Mengecek apakah stack kosong dan jika top_1018 < 0 => berarti kosong..

```
boolean isEmpty_1018()
  {
        return (top_1018 < 0);
```

Gambar 2.1.2.2 Method isEmpty.

2.1.2.3 Saat objek dibuat, stack masih kosong dan top = -1 artinya belum ada data.

```
stackArray_2511531018()
  {
        top_1018 = -1;
```

Gambar 2.1.2.3 Konstruktor.

2.1.2.4 Fungsi untuk menambahkan data ke stack. Cek apakah data penuh, jika tidak penuh top naik, lalu data dimasukkan. Tampilkan pesan.

```
boolean push_1018(int x)
  {
        if (top_1018 >= (MAX_1018 - 1)) {
              System.out.println("Stack overflow");
              return false;
        }
        else {
```

```
                a_1018[++top_1018] = x;
                System.out.println(x + " dimasukkan
dalam stack");
                return true;
```

Gambar 2.1.2.4 Method push (menambah data).

2.1.2.5 Mengambil dan menghapus data paling atas. Jika kosong, tampilkan Stack Underflow. Jika ada isi, ambil data, lalu top turun.

```
int pop_1018()
  {
        if (top_1018 < 0) {
                System.out.println("Stack Underflow");
                return 0;
        }
        else {
                int x = a_1018[top_1018--];
                return x;
```

Gambar 2.1.2.5 Method pop (menghapus data).

2.1.2.6 Melihat data paling atas tanpa menghapus.

```
int peek_1018()
  {
        if (top_1018 < 0) {
                System.out.println("Stack Underflow");
                return 0;
        }
        else {
                int x = a_1018[top_1018];
                return x;
```

Gambar 2.1.2.6 Method peek (melihat data teratas).

2.1.2.7 Menampilkan isi stack dari atas ke bawah. Cetak dari elemen paling atas sampai bawah.

```
void print() {
        for(int i = top_1018; i>-1; i--) {
                System.out.print(" "+ a_1018[i]);
        }
```

Gambar 2.1.2.7 Method print.

## 2.2 stackArrayDriver_2511531018

### 2.2.1. Kode Program

```
package pekan3_2511531018;

public class stackArrayDriver_2511531018 {
   public static void main(String[] args) {
```

```
        stackArray_2511531018 s = new
stackArray_2511531018();
        s.push_1018(10);
        s.push_1018(20);
        s.push_1018(30);
        System.out.println(s.pop_1018() + " dikeluarakan
dari stack");
        System.out.println("Elemen teratas adalah :" +
s.peek_1018());
        System.out.print("Elemen pada stack :");
        s.print();
    }

}
```

Gambar 2.2.1 Kode program stackArrayDriver_2511531018.

### 2.2.2. Langkah Kerja

2.2.2.1 Membuat objek s dari class stack dan semua operasi stack dilakukan lewat s.

```
stackArray_2511531018 s = new stackArray_2511531018();
```

Gambar 2.2.2.1 Membuat objek Stack.

2.2.2.2 Menambahkan data ke stack secara berurutan.

```
s.push_1018(10);
s.push_1018(20);
s.push_1018(30);
```

Gambar 2.2.2.2 Push (menambah data).

2.2.2.3 Menghapus data paling atas (yaitu 30) dan ditampilkan dilayar.

```
System.out.println(s.pop_1018() + " dikeluarakan dari
stack");
```

Gambar 2.2.2.3 Pop (menghapus data teratas).

2.2.2.4 Melihat elemen paling atas tanpa menghapus. Hasilnya sekarang = 20.

```
System.out.println("Elemen teratas adalah :" +
s.peek_1018());
```

Gambar 2.2.2.4 Peek (melihat elemen teratas).

2.2.2.5 Menampilkan seluruh isi stack dari atas ke bawah.

```
System.out.print("Elemen pada stack :");
        s.print();
```

Gambar 2.2.2.5 Print (menampilkan semua data)..

### 2.2.3. Output

```
10 dimasukkan dalam stack
20 dimasukkan dalam stack
30 dimasukkan dalam stack
30 dikeluarakan dari stack
Elemen teratas adalah :20
Elemen pada stack : 20 10
```

Gambar 2.2.3 Hasil output.

## 2.3 StackPostFix_2511531018

### 2.3.1 Kode Program

```
package pekan3_2511531018;

import java.util.Scanner;
import java.util.Stack;

public class StackPostFix_2511531018 {
   public static int postfixEvaluate(String expression) {
         Stack<Integer> s_1018 = new Stack<Integer>();
         Scanner input_1018 = new Scanner (expression);
         while (input_1018.hasNext()) {
               if (input_1018.hasNextInt()) {  // an operand
(integer)
                     s_1018.push(input_1018.nextInt());
               } else {
                     String operator = input_1018.next();
                     int operand2 = s_1018.pop();
                     int operand1 = s_1018.pop();
                     if (operator.equals("+")) {
                           s_1018.push(operand1 +
operand2);
                     } else if (operator.equals("-")) {
                           s_1018.push(operand1 -
operand2);
                     } else if (operator.equals("*")) {
                           s_1018.push(operand1 *
operand2);
                     } else {
                           s_1018.push(operand1 /
operand2);

                     }
               }
         }
         input_1018.close();
         return s_1018.pop();
         }
   public static void main (String[] args) {
```

```
        System.out.println("hasil postfix= "
+postfixEvaluate("5 2 4 * + 7 -"));
   }

}
```

Gambar 2.3.1 Kode program StackPostFix_2511531018.

### 2.3.2 Langkah Kerja

2.3.2.1 Method ini menerima string ekspresi postfix dan mengembalikan hasil perhitungan (int)..

```
public static int postfixEvaluate(String expression) {
```

Gambar 2.3.2.1 Method postfixEvaluate..

2.3.2.2 Menggunakan class Stack bawaan Java untuk menyimpan angka sementara saat perhitungan.

```
Stack<Integer> s_1018 = new Stack<Integer>();
```

Gambar 2.3.2.2 Membuat Stack.

2.3.2.3 Scanner dipakai untuk membaca isi string per kata, dipisahkan berdasarkan spasi.

```
Scanner input_1018 = new Scanner (expression);
```

Gambar 2.3.2.3 Scanner untuk membaca string.

2.3.2.4 Selama masih ada isi di ekspresi => lanjut proses.

```
while (input_1018.hasNext()) {
```

Gambar 2.3.2.4 Perulangan membaca token.

2.3.2.5 Jika yang dibaca adalah angka => langsung dimasukkan ke stack.

```
if (input_1018.hasNextInt()) {  // an operand
(integer)

  s_1018.push(input_1018.nextInt());
```

Gambar 2.3.2.5 Jika angka (Operand).

2.3.2.6 Ambil operator dan ambil 2 angka dari stack, operand2 => yang terakhir masuk dan operand1 => yang sebelumnya.

```
} else {
                        String operator =
input_1018.next();
                        int operand2 = s_1018.pop();
                        int operand1 = s_1018.pop();
```

Gambar 2.3.2.6 Jika operator (+, -, /).

2.3.2.7 Dilakukan operasi sesuai operator dan hasilnya dimasukkan kembali ke stack. Operator yang didukung => penjumlahan (+), pengurangan (-), perkalian (*), dan pembagian (/).

```
if (operator.equals("+")) {
                                s_1018.push(operand1 +
operand2);
                        } else if (operator.equals("-"))
{
                                s_1018.push(operand1 -
operand2);
                        } else if (operator.equals("*"))
{
                                s_1018.push(operand1 *
operand2);
                        } else {
                                s_1018.push(operand1 /
operand2);
```

Gambar 2.3.2.7 Proses perhitungan.

2.3.2.8 Hasil akhir ada di stack, diambil dan dikembalikan. Memanggil fungsi dengan ekspresi postfix dan menampilkan hasilnya.

```
input_1018.close();
        return s_1018.pop();
        }
  public static void main (String[] args) {
        System.out.println("hasil postfix= "
+postfixEvaluate("5 2 4 * + 7 -"));
```

Gambar 2.3.2.8 Setelah selesai dan Method main.

### 2.3.3. Output

```
hasil postfix= 6
```

Gambar 2.3.3 Hasil output.

## 2.4 NilaiMaksimum_2511531018

### 2.4.1 Kode Program

```
package pekan3_2511531018;

import java.util.Stack;

public class NilaiMaksimum_2511531018 {
```

```
   public static int max_1018(Stack<Integer> s_1018) {
          Stack<Integer> backup = new Stack<Integer>() ;
          int max_1018Value = s_1018.pop();
          backup.push(max_1018Value);
          while (!s_1018.isEmpty()) {
                 int next = s_1018.pop();
                 backup.push(next);
                 max_1018Value = Math.max(max_1018Value,
next);
          }
          while (!backup.isEmpty()) {
                 s_1018.push(backup.pop());
          }
          return max_1018Value;
   }
   public static void main(String[] args) {
          Stack<Integer> s_1018 = new Stack<Integer> ();
          s_1018.push(70);
          s_1018.push(12);
          s_1018.push(20);
          System.out.println("isi stack "+s_1018);
          System.out.println("Stack teratas "+s_1018.peek());
          System.out.println("Nilai maksimum
"+max_1018(s_1018));

   }

}
```

Gambar 2.4.1 Kode program NilaiMaksimum_2511531018.

### 2.4.2 Langkah Kerja

2.4.2.1 Method ini menerima stack dan mengembalikan nilai maksimum dari stack.

```
public static int max_1018(Stack<Integer> s_1018) {
```

Gambar 2.4.2.1 Method max_1018.

2.4.2.2 Digunakan untuk menyimpan sementara isi stack agar data asli bisa dikembalikan lagi nanti.

```
Stack<Integer> backup = new Stack<Integer>() ;
```

Gambar 2.4.2.2 Membuat stack backup.

2.4.2.3 Ambil elemen pertama sebagai nilai maksimum sementara dan simpan juga ke backup.

```
int max_1018Value = s_1018.pop();
        backup.push(max_1018Value);
```

Gambar 2.4.2.3 Ambil nilai awal.

2.4.2.4 Ambil semua isi stack satu per satu, bandingkan tiap nilai dengan nilai maksimum saat ini.

```
while (!s_1018.isEmpty()) {
                int next = s_1018.pop();
                backup.push(next);
                max_1018Value = Math.max(max_1018Value,
next);
```

Gambar 2.4.2.4 Loop mencari nilai maksimum.

2.4.2.5 Isi stack dikembalikan seperti semula, lalu setelah fungsi selesai, stack tidak berubah.

```
while (!backup.isEmpty()) {
                s_1018.push(backup.pop());
```

Gambar 2.4.2.5 Mengembalikan isi stack.

2.4.2.6 Return nilai maksimum.

```
return max_1018Value;
  }
```

Gambar 2.4.2.6 Return nilai maksimum.

2.4.2.7 Membuat stack, lalu isi: s_1018.push(70);, s_1018.push(12);, dan s_1018.push(20);.

```
Stack<Integer> s_1018 = new Stack<Integer> ();
        s_1018.push(70);
        s_1018.push(12);
        s_1018.push(20);
```

Gambar 2.4.2.7 Method main.

2.4.2.8 Menampilkan isi stack, menampilkan elemen teratas, dan menampilkan nilai maksimum.

```
System.out.println("isi stack "+s_1018);
        System.out.println("Stack teratas
"+s_1018.peek());
        System.out.println("Nilai maksimum
"+max_1018(s_1018));
```

Gambar 2.4.2.8 Output.

### 2.4.3 Ouput

```
isi stack [70, 12, 20]
Stack teratas 20
Nilai maksimum 70
```

Gambar 2.4.3 Hasil output.

## 2.5 SiswaStack_2511531018

### 2.5.1 Kode Program

```
package pekan3_2511531018;

import java.util.ArrayList;

class Siswa {
   String nama;
   int nim;

   public Siswa(String nama, int nim) {
         this.nama = nama;
         this.nim = nim;
   }

   @Override
   public String toString() {
         return "Nim " + nim + ", Nama: " + nama;
   }
}

public class SiswaStack_2511531018 {
   private ArrayList<Siswa> stack;

   public SiswaStack_2511531018() {
         stack = new ArrayList<>();
   }

   public void push(Siswa mhs) {
         stack.add(mhs);
   }
   public Siswa pop() {
         if (!isEmpty()) {
               return stack.remove(stack.size() - 1);
         }
         return null;
   }
   public Siswa peek() {
         if (!isEmpty()) {
               return stack.get(stack.size() - 1);
         }
         return null;
   }

   public boolean isEmpty() {
         return stack.isEmpty();
   }

   public void tampilkanSiswa() {
         for (int i = stack.size() - 1; i >= 0; i--) {
               System.out.println(stack.get(i));
         }
   }
   public static void main(String[] args) {
```

```
        SiswaStack_2511531018 studentStack = new
SiswaStack_2511531018();

        Siswa mhs1 = new Siswa("Ali", 1);
        Siswa mhs2 = new Siswa("Boby", 2);
        Siswa mhs3 = new Siswa("Charles", 3);
        studentStack.push(mhs1);
        studentStack.push(mhs2);
        studentStack.push(mhs3);

        System.out.println("Siswa di dalam stack:");
        studentStack.tampilkanSiswa();

        System.out.println("siswa teratas " +
studentStack.peek());
        System.out.println("mengeluarkan siswa teratas dari
stack: " + studentStack.pop());
        System.out.println("daftar siswa setelah di pop :"
);
        studentStack.tampilkanSiswa();
    }

}
```

Gambar 2.5.1 Kode program SiswaStack_251131018.

### 2.5.2 Langkah Kerja

2.5.2.1 Menyimpan nama siswa dan menyimpan nomor induk mahasiswa.

```
class Siswa {
  String nama;
  int nim;
```

Gambar 2.5.2.1 Class Siswa.

2.5.2.2 Digunakan untuk membuat objek siswa dan mengisi atribut.

```
public Siswa(String nama, int nim) {
        this.nama = nama;
        this.nim = nim;
```

Gambar 2.5.2.2 Konstruktor.

2.5.2.3 Mengubah objek jadi teks saat dicetak, sehingga outputnya rapi.

```
@Override
  public String toString() {
        return "Nim " + nim + ", Nama: " + nama;
```

Gambar 2.5.2.3 Method toString().

2.5.2.4 ArrayList digunakan sebagai wadah stack dan menyimpan objek siswa.

```
private ArrayList<Siswa> stack;
```

Gambar 2.5.2.4 Class SiswaStack.

2.5.2.5 Membuat ArrayList kosong saat objek dibuat.

```
public SiswaStack_2511531018() {
        stack = new ArrayList<>();
```

Gambar 2.5.2.5 Konstruktor.

2.5.2.6 Menambahkan siswa ke stack (bagian atas)..

```
public void push(Siswa mhs) {
        stack.add(mhs);
```

Gambar 2.5.2.6 Method push.

2.5.2.7 Mengambil dan menghapus elemen terakhir, jika kosong => return null.

```
public Siswa pop() {
        if (!isEmpty()) {
                return stack.remove(stack.size() - 1);
        }
        return null;
```

Gambar 2.5.2.7 Method pop.

2.5.2.8 Melihat elemen paling atas pada stack tanpa menghapus elemen tersebut.

```
public Siswa peek() {
        if (!isEmpty()) {
                return stack.get(stack.size() - 1);
        }
        return null;
```

Gambar 2.5.2.8 Method peek.

2.5.2.9 Mengecek apakah stack kosong.

```
public boolean isEmpty() {
        return stack.isEmpty();
```

Gambar 2.5.2.9 Method isEmpty.

2.5.2.10 Menampilkan isi stack dari atas ke bawah menggunakan get(1).

```
public void tampilkanSiswa() {
        for (int i = stack.size() - 1; i >= 0; i--) {
                System.out.println(stack.get(i));
```

Gambar 2.5.2.10 Method tampilkanSiswa.

2.5.2.11 Membuat stack dan menampilkan data.

```
SiswaStack_2511531018 studentStack = new
SiswaStack_2511531018();
```

```
        Siswa mhs1 = new Siswa("Ali", 1);
        Siswa mhs2 = new Siswa("Boby", 2);
        Siswa mhs3 = new Siswa("Charles", 3);
        studentStack.push(mhs1);
        studentStack.push(mhs2);
        studentStack.push(mhs3);

        System.out.println("Siswa di dalam stack:");
        studentStack.tampilkanSiswa();

        System.out.println("siswa teratas " +
studentStack.peek());
        System.out.println("mengeluarkan siswa
teratas dari stack: " + studentStack.pop());
        System.out.println("daftar siswa setelah di
pop :" );
        studentStack.tampilkanSiswa();
```

Gambar 2.5.2.11 Method main dan tampilkan.

### 2.5.3 Output

```
Siswa di dalam stack:
Nim 3, Nama: Charles
Nim 2, Nama: Boby
Nim 1, Nama: Ali
siswa teratas Nim 3, Nama: Charles
mengeluarkan siswa teratas dari stack: Nim 3, Nama: Charles
daftar siswa setelah di pop :
Nim 2, Nama: Boby
Nim 1, Nama: Ali
```

## 2.6 Tugas

### 2.6.1 Kode Program Website_2511531018

```
package TugasPekan3;

public class Website_2511531018 {

       private String judul_1018;
       private String url_1018;

       public Website_2511531018(String judul_1018, String
url_1018) {
           this.judul_1018 = judul_1018;
           this.url_1018 = url_1018;
       }
       public String getJudul_1018() {
           return judul_1018;
       }
       public String getUrl_1018() {
           return url_1018;
       }
```

```
        public void setJudul_1018(String judul_1018) {
            this.judul_1018 = judul_1018;
        }

        public void setUrl_1018(String url_1018) {
            this.url_1018 = url_1018;
        }
        @Override
        public String toString() {
            return "Judul: " + judul_1018 + "\nURL: " +
url_1018;
        }
    }
```

Gambar 2.6.1 Kode Program Website_2511531018.

### 2.6.2 Kode Program Browser_2511531018

```
package TugasPekan3;

import java.util.Scanner;
import java.util.Stack;

public class Browser_2511531018 {
    public static void main(String[] args) {
        Stack<Website_2511531018> history = new Stack<>();
        Scanner sc = new Scanner(System.in);
        int pilihan;

        do {
            System.out.println("\n=== Browser History NIM:
2511531018 ===");
            System.out.println("1. Kunjungi Website (Push)");
            System.out.println("2. Tombol Back (Pop)");
            System.out.println("3. Lihat Halaman Aktif
(Peek)");
            System.out.println("4. Cek Status History");
            System.out.println("5. Keluar");
            System.out.print("Pilihan: ");

            try {
                pilihan = sc.nextInt();
                sc.nextLine();
            } catch (Exception e) {
                System.out.println("Input harus angka!");
                sc.nextLine();

                pilihan = 0;
            }

            switch (pilihan) {
                case 1:
                    // PUSH
                    System.out.print("Masukkan Judul: ");
                    String judul = sc.nextLine();
                    System.out.print("Masukkan URL: ");
                    String url = sc.nextLine();
```

```
                    history.push(new
Website_2511531018(judul, url));
                    System.out.println("Berhasil mengunjungi
halaman!");
                    break;

                case 2:
                    // POP
                    if (!history.isEmpty()) {
                        Website_2511531018 removed =
history.pop();
                        System.out.println("Kembali dari: " +
removed.getJudul_1018());
                    } else {
                        System.out.println("History kosong,
tidak bisa kembali!");
                    }
                    break;

                case 3:
                    // PEEK
                    if (!history.isEmpty()) {
                        Website_2511531018 current =
history.peek();
                        System.out.println("Halaman saat
ini:");
                        System.out.println(current);
                    } else {
                        System.out.println("Tidak ada halaman
yang dibuka.");
                    }
                    break;

                case 4:
                    // STATUS
                    if (history.isEmpty()) {
                        System.out.println("History
kosong.");
                    } else {
                        System.out.println("Jumlah history: "
+ history.size());
                    }
                    break;

                case 5:
                    System.out.println("Keluar dari
program.");
                    break;

                default:
                    System.out.println("Pilihan tidak
valid.");
            }

        } while (pilihan != 5);
```

```
        sc.close();
    }
}
```

Gambar 2.6.2 Kode Program Browser_2511531018.

### 2.6.3 Output

1. Mengunjungi website

```
=== Browser History NIM: 2511531018 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 1
Masukkan Judul: ilearn
Masukkan URL: itilearn.ac.id
Berhasil mengunjungi halaman!

=== Browser History NIM: 2511531018 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 1
Masukkan Judul: sso
Masukkan URL: sso.unand.ac.id
Berhasil mengunjungi halaman!

=== Browser History NIM: 2511531018 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 1
Masukkan Judul: shopee
Masukkan URL: shopee.com.id
Berhasil mengunjungi halaman!
```

Pada gambar diatas menunjukkan bahwa user sedang mengunjungi website ilearn, sso, dan shopee secara berurutan. Website pertama yang dikunjungi adalah ilearn, setelah ilearn user membuka website sso, dan setelah itu user membuka website shopee.

2. Melihat halaman yang sedang aktif

```
=== Browser History NIM: 2511531018 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 3
Halaman saat ini:
Judul: shopee
URL: shopee.com.id
```

Ketika user memilih tombol untuk melihat halaman aktif, user dapat melihat halaman yang sedang aktif yang dimana halaman tersebut merupakan website yang terakhir dikunjungi oleh user.

3. Tombol back

```
=== Browser History NIM: 2511531018 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 2
Kembali dari: shopee
```

Ketika user memilih tombol back, maka halaman terakhir yang dikunjungi akan dihapus dari daftar histori. Lalu halaman akan beralih ke halaman sebelum halaman yang telah dihapus.

4. Cek status history

```
=== Browser History NIM: 2511531018 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 4
Jumlah history: 1
```

Cek status histori berguna untuk melihat berapa halaman yang masi tersimpan dalam histori. Output yang dihasilkan berupa angka.

5. Keluar

```
=== Browser History NIM: 2511531018 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 5
Keluar dari program.
```

Tombol keluar berguna untuk user keluar dari program.

# BAB III PENUTUP

## KESIMPULAN

Program ini mengimplementasikan struktur data stack menggunakan ArrayList untuk menyimpan objek Siswa yang berisi nama dan NIM di Java. Stack bekerja dengan prinsip LIFO *(Last In First Out)*, di mana data terakhir yang dimasukkan akan menjadi yang pertama dikeluarkan. Program menyediakan operasi dasar seperti push untuk menambah data, pop untuk mengambil sekaligus menghapus data teratas, peek untuk melihat data teratas, serta fungsi untuk menampilkan seluruh isi stack. Namun, terdapat kesalahan pada method peek() karena menggunakan remove() yang menyebabkan data ikut terhapus, sehingga seharusnya diperbaiki dengan get() agar hanya melihat tanpa mengubah isi stack.

# DAFTAR PUSTAKA

[1] Robert Lafore, *Data Structures and Algorithms in Java*, 2nd ed. Indianapolis, IN, USA: Sams Publishing, 2003.

[2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.